

Iverto Backend: Combined Load Test Report (30 / 100 / 300 Users)

Date: 26 Sep 2026, 10:01 – 10:32 IST
Target: `https://api.iverto.ai/devhostel1`, Hostel **v2** API (`/v2/tenants/acme/...`), tenant `ACME`
Tool: k6 v2.2.0 (`k6/v2/users.js`), one Windows laptop, one public IP
Traffic: reads only, role mix student 80% · guardian 15% · warden 4% · admin 1%
Per-test reports: `REPORT-2026-09-26-30-users.pdf`, `-100-users.pdf`, `-300-users.pdf`

1. Summary

The backend serves at most ~30 requests per second, whatever the number of users. That is enough for about 30 people using the app at once. Past that, every request waits in a queue: at 100 users a screen takes ~2 s to load, and at 300 users opening the app takes **over a minute**. Warden and admin screens (dashboard, roster, groups) already fail at 30 users and fail **100%** at 300.

	30 users	100 users	300 users
Verdict	⚠ Pass for students, staff screens fail	✗ Fail: slow for everyone	✗ Fail: unusable
Throughput (req/s)	23.8	27.2	29.9
App opens per second	2.65	3.03	3.03
Latency p50	118 ms	2.0 s	7.7 s
Latency p95 (SLO 800 ms)	636 ms ✓	5.8 s ✗	13.9 s ✗
Latency p99 (SLO 2 s)	1.26 s ✓	8.6 s ✗	15.8 s ✗
One app-open, p50 / p95	9 s / 14 s	28 s / 47 s	82 s / 103 s
HTTP errors (SLO < 1%)	0.36% ✓	0.69% ✓	0.58% ✓
<code>/dashboard</code> success	27%	2%	0%
<code>/roster</code> success	0%	12%	0%
<code>/groups</code> success	100%	0%	0%
<code>/app-config</code> p50 (a constant)	95 ms	1.8 s	7.2 s
Server crashed?	No	No	No (healthy 0.4 s after)

The HTTP error rate looks healthy in every test. **It isn't.** Student requests don't fail, they wait: 7–16 s each at 300 users. A real person gives up long before then.

2. What the three tests show together

2.1 A hard ceiling at ~30 req/s

users	30		23.8 req/s	p50 0.1 s
users	100		27.2 req/s	p50 2.0 s
users	300		29.9 req/s	p50 7.7 s

10× the users gave **+26% throughput** and **65× the median latency**. The system is saturated at ~30 users. This matches the 24 Sep stress test, which found the knee at ~30–35 req/s.

2.2 One shared bottleneck, most likely the database connection pool

- At 30 users, endpoints have clearly different costs: `/app-config` 95 ms, `/me/student` 405 ms, `/dashboard` 12.8 s.
- At 300 users, they are **almost the same** (7–9 s p50), cheap or expensive. Requests wait for the same slot, then run quickly.
- On 24 Sep, `/app-config` alone served ~194 req/s with p95 316 ms. So Caddy, TLS, the network and Node itself can go ~6× faster than the ceiling. **What saturates is DB-backed work.**
- Staff `500`s arrive at 12–22 s, which fits a server-side statement or pool-acquire timeout, not a crash.
- Each burst of `/dashboard` / `/roster` calls lines up with a throughput dip (to 12–18 req/s) in all three tests. Heavy staff queries hold connections for 10–30 s and starve everyone else.

2.3 Staff screens are broken before load is even a factor

At 30 users, with only 3 wardens and 2 admin tokens in the pool, `/dashboard` took 10–20 s and failed 73% of the time. `/roster` never succeeded. These endpoints have a query problem, not just a capacity problem.

2.4 The server degrades gracefully in this model, but that won't hold

No `502`s and no crash in any test. The service answered in 0.4 s straight after the 300-user run. That's because these simulated users wait for each response before sending more. Real users force-close and reopen, and the app retries. On 24 Sep, an open-model test at 30 app-opens/s led to 30 s timeouts, Caddy `502`s and a **2.5-minute outage**. Expect that behaviour in a real pre-curfew rush.

3. What went wrong, and how to fix it

Ordered by impact. **P0** = blocks use at the target of 500–2,000 users.

P0-1 • `/dashboard`, `/roster`, `/groups` fail even at low load

Seen in	Evidence
30 users	dashboard 73% <code>500</code> (p50 12.8 s), roster 100% fail (30 s), groups 5.8 s p50
100 users	dashboard 98% <code>500</code> , roster 88% timeout, groups 100% <code>500</code>
300 users	all three 100% failed

Cause: unindexed or N+1 queries that compute everything per request (single-user `/dashboard` was 4–5.5 s and `/roster` 7–9 s on 24 Sep, with no load).

Fix:

- `EXPLAIN (ANALYZE, BUFFERS)` the queries behind each of the three.
- Dashboard:** one `GROUP BY` aggregate over tenant/site, cached 15–30 s in Redis or a materialized view. Target < 100 ms. Also fix its count mismatch (`total ≠ inside + outside`).
- Roster:** paginate in SQL (keyset on an indexed column) instead of in memory. Indexes `(tenant_id, site_id)`, `(tenant_id, status)`. Target < 300 ms for `limit=20`.
- Groups / memberships / roles:** load members with one join, index `(group_id)`.
- `statement_timeout` ≈ 5 s, so a slow report fails fast instead of holding a connection for 20 s.

P0-2 · Throughput ceiling ~30 req/s: every request queues

Seen in	Evidence
30 users	already at ~28–30 req/s steady state, no headroom
100 users	+14% throughput, p50 17× worse (2 s)
300 users	~30 req/s, p50 7.7 s, one app-open 82 s

Cause: DB connection pool exhausted by slow queries (P0-1) plus slow student queries (`/me/student` 405 ms p50 even at 30 users, `/me/children` 707 ms).

Fix:

1. **Confirm with server metrics** for 10:05–10:32 IST today: pool waiting count, active DB connections, `pg_stat_activity`, CPU, event-loop lag.
2. **Index the student and guardian paths:** `(tenant_id, student_id)`, `(tenant_id, site_id)`, `(tenant_id, status, created_at)` on permissions, `(student_id) / (guardian_id)` on the guardian link table. Remove the guardian N+1 in `/me/student` and `/me/children*`. Target < 50 ms each.
3. **Isolate staff reports** on their own small pool or a read replica so they cannot starve students.
4. **Size the pool explicitly** (20–30, below Postgres `max_connections`) with a 1–2 s acquire timeout.
5. **Scale out** after the query fixes: ≥ 2 app instances behind Caddy with `health_uri` checks and `lb_try_duration`.

P0-3 · Static data goes through the database on every app open

`/app-config`, `/me/branding`, `/sites`, `/categories` and `/me/notifications/unread-count` make up ~35% of all requests. In the 300-user test they each waited 7+ s for data that almost never changes.

Fix: cache them in memory or Redis on the server, and send `ETag / Cache-Control: max-age` so the app can skip or revalidate them. Batch `unread-count` into an existing call, or push it over the socket once socket auth works (24 Sep report, P0-2).

P1-4 · No back-pressure: users wait instead of being told to retry

At saturation the server queues requests for up to 30 s. Users will retry, which adds even more load (the 24 Sep collapse).

Fix: return a fast `503 + Retry-After` when the pool queue is full. In the app, add a request timeout of ~10 s, exponential backoff, and a "busy, retrying" state instead of an indefinite spinner.

P1-5 · Test data

`parent.acm0001@example.com` fails login in every pool (two guardians share one email). Make principal email unique per tenant and give the second guardian their own address.

Still open from the 24 Sep report (not re-tested today)

- App 1.2.2 logs in via v1 routes that `/devhoste1` doesn't serve (**404**). Release blocker.
- Socket.IO `/mobile` rejects v2 tokens, so there are no realtime updates.
- Login/refresh limit of 10/min per IP will lock out a campus behind one NAT.
- Staff MFA (`x-mfa-level: 1`) not enforced.

4. Capacity in plain terms

	Today	After P0-1 ... P0-3 (target)
Concurrent active users with p95 < 800 ms	~30	300+
Sustained throughput	~30 req/s	≥ 100 req/s (≥ 300 req/s for the 2,000-user busy hour)
Staff dashboard	fails at 30 users	< 100 ms, 100% 2xx
App open time at 300 users	82 s	< 2 s

A 500–2,000-student hostel before curfew means hundreds of people opening the app within minutes. **Today's backend handles about 30.**

5. Suggested order of work

1. Pull server/DB metrics for today's test window and confirm pool exhaustion (half a day).
2. P0-1: fix dashboard, roster and groups queries plus `statement_timeout` (largest single gain, removes all 500 s).
3. P0-2: student/guardian indexes and N+1 removal, separate report pool.
4. P0-3: cache static endpoints.
5. P1-4: fast 503, plus app timeouts and backoff.
6. Re-run all three tests with the same command. Pass criteria **at 300 users**: p95 < 800 ms, errors < 1%, staff endpoints 100% 2xx.

6. How the tests were run

Item	Value
Script	<code>load-test/k6/v2/users.js</code> (closed model: ramp 1 min, hold 4 min, ramp down 30 s; each user = one role journey then 3–8 s idle)
Runner	<code>load-test/results/2026-09-26/run-users.sh 30 100 300</code> (health check → fresh 35-token pool → k6 → CSV analysis)
Tokens	35 accounts per test (24 students, 6 guardians, 3 wardens, 2 admins), re-issued before each test because tokens live 15 min and login is capped at 10/min/IP. 0 × 429 in every pool.
Raw data	<code>load-test/results/2026-09-26/users-{30,100,300}. {csv,json,log}</code> and <code>*.csv.analysis.json</code>

Caveats

- One laptop and one IP generated all the load. On 24 Sep the same setup pushed ~200 req/s to a cheap endpoint, so the 30 req/s ceiling is the server's, not the generator's.
- 35 tokens were shared by up to 300 virtual users. Real distinct users would put more load on the DB, not less.
- Reads only. Writes (pass requests, approvals, late entries) would add load on top.
- The closed model under-represents the retry storm real users create (see §2.4).